Verification workshop: OpenJML and JBMC

In the lecture we saw that the weakest precondition of a loop is an infinite formula, and two ways out of it:

	A · Loop invariants	**B · Unrolling (bounded model checking)**
Tool	OpenJML (Java + JML)	JBMC (Java + `assert`)
You provide	contracts and loop invariants	a harness and a bound *K*
"OK" means	correct for all executions	no bug in executions within the bound
"Error" means	a bug, or an invariant that is too weak	a bug, with a concrete counterexample

In this workshop you use both tools on the **same programs**, written in **Java 21** (records, enums, sealed interfaces, pattern matching), and compare what each one asks from you and what each one gives you back.

The workshop has three parts. Each folder has its exercises (`README.md`), the code, and an `ANSWERS.md` for your answers; questions are marked **Q**, and exercises marked ★ are optional.

Part	**What**	**Deadline**
`hw/`	**Homework**: the lecture's examples (abs, sumx, max, binary search) with both tools	November 17, 2026
`plus/`	**Homework plus**: invariants from an LLM, the Zune bug, and classes with state (a record, a stack, a state machine)	November 24, 2026
`bonus/`	**Bonus** (optional): a data structure on the heap, and a proof for any number of events	December 7, 2026

Start with `hw/`: it introduces everything the other parts use.

Get your own copy of this repo

1. Sign in to GitHub and click **Fork** (top right of this page). Pick your account as the owner and keep the repo name. Working in a group? Only one member forks, and adds the others in the fork under **Settings → Collaborators**.
2. Clone **your fork**, not this repo, and work inside it:

```
git clone https://github.com/<your-user>/verification-workshop.git
cd verification-workshop
```

Forks of this repo are public. Write your own solution: copying from someone else's fork is plagiarism.

Setup

You need a JDK 21 or newer (`javac`), JBMC, python3, and about 1 GB of disk. OpenJML is downloaded into `tools/openjml` by `./setup.sh` .

- **macOS**: `brew install cbmc` (it includes `jbmc`) and, if you have no JDK 21+, `brew install openjdk@21` . Then `./setup.sh` .
- **Ubuntu 22.04 / 24.04 (x86-64)**: `sudo apt install openjdk-21-jdk python3 curl unzip` . Download the CBMC `.deb` for your Ubuntu version from the CBMC releases (it includes `jbmc`) and install it with `sudo apt install ./ubuntu-24.04-cbmc-*-Linux.deb` . Then `./setup.sh` .
- **Windows**: install WSL 2 with Ubuntu 24.04 and follow the Ubuntu instructions inside it.

Check that everything works:

```
./esc hw/Constant.java                                    # OK: all proof obligations
        verified
./bmc MaxCheck.check --unwind 3 --no-unwinding-assertions   # runs (and finds a bug:
        see hw, exercise 5)
```

The two commands

`./esc folder/File.java` — OpenJML

Checks every method of the file against its JML contract. Use `--method NAME` to check one method. Each problem is reported as

```
hw/SumX.java:16: verify: The prover cannot establish an assertion (Postcondition:
hw/SumX.java:6:) in method sum
        return s;
        ^
hw/SumX.java:6: verify: Associated declaration: hw/SumX.java:16:
    //@ ensures \result == x * (x + 1) / 2;
        ^
```

(the postcondition of line 6 may not hold at the `return` of line 16).

The word in parentheses says which proof obligation failed: `Postcondition` , `LoopInvariant` (not preserved by an iteration), `LoopInvariantBeforeLoop` (not true on entry), `LoopDecreases` , `ArithmeticOperationRange` (an `int` overflow), `PossiblyNegativeIndex` , `PossiblyTooLargeIndex` , `Precondition` (of a called method), `InvariantExit` (a class invariant broken by a method), …

JML reminder: `//@ requires` , `//@ ensures` (`\result` , `\old(e)`), `//@ loop_invariant` , `//@ decreases` , `//@ assignable` , `//@ invariant` (class invariant), `/*@ pure @*/` , quantifiers `(\forall int k; range; body)` , `(\exists ...)` , `(\num_of ...)` , `==>` , `<==>` . Specifications are evaluated with mathematical integers, the code with 32-bit `int` . See the OpenJML tutorial.

`./bmc Class.method --unwind K` — JBMC

Compiles all the `.java` files of the folder (`hw/` , `plus/` or `bonus/`) that has `Class.java` (a compile error in any of them stops it) and runs JBMC on a static method, the **harness**. The harness parameters are the inputs: JBMC considers *all* their values at once, encoding the program as a SAT formula (lecture:

"from code to a formula").

```
public static void check(int[] a) {
    CProver.assume(a != null && a.length > 0);   // the precondition: other inputs are
        ignored
    int r = Max.max(a);
    for (int j = 0; j < a.length; j++)
        assert r >= a[j];                         // the postcondition
}
```

`CProver.assume(c)` is the `assume` statement of the lecture; `CProver.nondetInt()` / `nondetBoolean()` return an arbitrary value. Besides your `assert`s, JBMC checks array bounds, null dereferences, division by zero, casts and uncaught exceptions.

Options:

option	meaning
`--unwind K`	unroll every loop *K* times (required)
`--no-unwinding-assertions`	after *K* iterations, `assume !B`: longer executions are ignored. Without it, `assert !B`: JBMC also checks that *K* is enough (lecture: "assume !B or assert !B?")
`--max-nondet-array-length N`	input arrays have length ≤ *N* (default 5)
`--property ID`	check only one property

The output lists the properties that fail, the size of the SAT formula, and a counterexample for each failure: the inputs JBMC chose, and the calls and assignments that lead to the failing line. `BMC_FULL=1 ./bmc ...` also lists the properties that hold; `BMC_RAW=1 ./bmc ...` shows JBMC's own output.

In VS Code

Open this folder in VS Code (on Windows, from WSL: `code .` inside Ubuntu). The repo comes with tasks for both tools (**Terminal › Run Task…**):

Task	What it runs
OpenJML: verify this file (**Cmd/Ctrl+Shift+B**)	`./esc` on the file in the editor
OpenJML: verify one method of this file	`./esc ... --method NAME` (asks for the name)
JBMC: check a harness	`./bmc` on a harness you pick from a list, with the options you type
Setup: install OpenJML and check the tools	`./setup.sh`

The output appears in the terminal panel, and the failures are listed in the **Problems** panel and underlined in the code (for OpenJML, the line that fails and, as info, the clause of the contract it violates). JBMC's counterexample is only in the terminal.

Install the recommended extension *Language Support for Java* (`redhat.java`, VS Code offers it when you open the folder) for highlighting, navigation and compile errors; the repo's settings tell it about the three source folders and JBMC's jar.

★ OpenJML also has its own VS Code extension (alpha): it verifies with **Cmd/Ctrl+E**, shows ✓ / ✗ above each method and underlines failures. Install it with `./setup.sh --vscode` (it sets `openjml.serverPath` in your VS Code user settings), then reload the window. The tasks above work without it.

Which Java?

Both tools understand Java 21 (records, enums, sealed interfaces, switch expressions on ints and enums, `instanceof` with a type pattern, `var`), with limits. JBMC only models part of the JDK: avoid lambdas, streams, collections (`ArrayList`, `HashMap`, …) and `switch` on types (`case Circle c ->`); a library method JBMC does not model returns an arbitrary value, which gives false counterexamples. OpenJML does not support record deconstruction patterns (`e instanceof EnterPin(int p)`; use `e instanceof EnterPin p` and `p.pin()`). All the code in this workshop stays inside what both tools support.

Submission

1. Commit and push to the `main` branch of your fork, as often as you like:

   ```
   git add -A
   git commit -m "hw: contracts and invariants for SumX and Max"
   git push
   ```

2. Submit the URL of your fork on Moodle (once: the same fork holds the three parts).

For each part we grade the last commit on `main` pushed before its deadline: homework (`hw/`), **November 17, 2026**; homework plus (`plus/`), **November 24, 2026**; bonus (`bonus/`, optional), **December 7, 2026**. We run both tools on your code and your harnesses, and read your `ANSWERS.md`. Don't weaken the contracts and harnesses we give you (no `//@ assume`, no `CProver.assume(false)`): a proof of a weaker property is not a proof.

Updates from the course

If we fix something in this repo, your fork's page says "This branch is N commits behind". Click **Sync fork** → **Update branch**, and then run `git pull` in your clone.